Collection de collections y excepciones

Contexto

Se solicita implementar un sistema de gestión de alumnos y cursos en Java utilizando un `HashMap<String,HashSet<Alumno>>`, donde la clave es el nombre del curso (tipo `String`) y el valor es una lista de alumnos (tipo `HashSet<Alumno>`). La clase `Alumno` tiene un atributo `DNI` que se utiliza para la comparación entre objetos mediante el método `equals`. Además, se deben crear excepciones personalizadas para manejar diversas situaciones del sistema.

Consignas

1. Agregar alumno a un curso

Crea un método `agregarAlumno(String curso, Alumno alumno)` que permita añadir un alumno a la lista de alumnos de un curso. Si el curso no existe, lanza la excepción `CursoNoEncontradoException`. Si el alumno ya está inscrito en el curso (verificado por el atributo `DNI`), lanza la excepción `AlumnoYaInscriptoException`.

---

2. Eliminar alumno de un curso

Crea un método `eliminarAlumno(String curso, String dni)` que permita eliminar a un alumno de un curso, identificándolo por su `DNI`. Si el curso no existe, lanza la excepción `CursoNoEncontradoException`. Si el alumno no está inscrito en el curso, lanza la excepción `AlumnoNoEncontradoException`.

---

3. Listar alumnos de un curso

Crea un método `listarAlumnos(String curso)` que devuelva una lista con los nombres de los alumnos inscritos en un curso. Si el curso no existe, lanza la excepción `CursoNoEncontradoException`. Si el curso está vacío, lanza la excepción `CursoVacioException`.

---

4. Contar alumnos en un curso

Crea un método `contarAlumnos(String curso)` que devuelva el número de alumnos inscritos en un curso. Si el curso no existe, lanza la excepción `CursoNoEncontradoException`. Si no hay alumnos en el curso, lanza la excepción `CursoVacioException`.

---

5. Transferir alumno entre cursos

Crea un método `transferirAlumno(String cursoOrigen, String cursoDestino, String dni)` que mueva a un alumno de un curso a otro utilizando su `DNI`. Si alguno de los cursos no existe, lanza `CursoNoEncontradoException`. Si el alumno no está inscrito en el curso de origen, lanza `AlumnoNoEncontradoException`. Si el alumno ya está inscrito en el curso de destino, lanza `AlumnoYaInscritoException`.

---

6. Listar todos los cursos disponibles

Crea un método `listarCursos()` que devuelva una lista con los nombres de todos los cursos disponibles. Si no hay cursos en el sistema, lanza la excepción `NoHayCursosException`.

---

7. Verificar si un alumno está inscrito en un curso

Crea un método `verificarInscripcion(String curso, String dni)` que devuelva `true` si un alumno con el `DNI` especificado está inscrito en el curso, y `false` en caso contrario. Si el curso no existe, lanza la excepción `CursoNoEncontradoException`.

---

8. Cambiar nombre de un curso

Crea un método `cambiarNombreCurso(String cursoActual, String nuevoNombre)` que cambie el nombre de un curso. Si el curso no existe, lanza la excepción `CursoNoEncontradoException`. Si ya existe un curso con el nuevo nombre, lanza `CursoYaExistenteException`.

Importante: Además de implementar las funcionalidades y excepciones solicitadas, organiza tu proyecto de manera correcta utilizando paquetes en Java. Esto te ayudará a mantener un código más estructurado y modular. La organización recomendada es la siguiente:

1. **`Modelo`:**
    - Este paquete contiene las **clases que modelan los objetos** principales del sistema, como `Alumno`. Estas clases encapsulan atributos y métodos relevantes para representar alumnos y cursos. En este caso, `Alumno` debería implementar el método `equals()` basándose en el atributo `DNI`.
2. **`Excepciones`:**
    - Agrupa todas las **excepciones personalizadas**, facilitando la gestión de errores específicos. Al centralizar las excepciones, permite un manejo claro de las posibles fallas o errores en el sistema, evitando duplicación de lógica y mejorando la legibilidad del código.
3. **`Gestion`:**
    - Aquí se implementa la lógica de negocio, principalmente en la clase `GestionCursos`, que es responsable de **gestionar el `HashMap`** que relaciona los nombres de los cursos (`String`) con los alumnos (`HashSet<Alumno>`). Todas las operaciones de agregar, eliminar, listar, y transferir alumnos se realizan en esta clase.
4. **`Menu`:**
    - Este paquete contiene la clase `Menu`, que será la responsable de la **interacción con el usuario**. Aquí se maneja la lógica de presentación, como la selección de opciones del menú y la interacción con la clase `GestionCursos` para ejecutar las operaciones.

5. **`Main:`**

★ Para el Main no crearemos un paquete de momento, pero si será parte de nuestro proyecto. La clase `Main` siempre será el **punto de entrada** del programa. Aquí se inicializa el menú y se configura todo lo necesario para arrancar la aplicación.